A Stress Test of LLM Syntactic Heads: Does High-Surprisal Input Collapse Dependency Grammar? A Cross-Model, Cross-Language Study

Himank Khandelwal(240450) · Virender(241171) · Sanchit Hamane(240925) · Kshitij Pramod Ramtekkar(240571)

Abstract. We investigate whether syntactic attention heads in two large language models - GPT-2 (117M) and Llama-3.2-3B (3B) - exhibit confidence collapse under high-surprisal input, across two typologically distinct languages: English (SVO) and Hindi (SOV). GPT-2 was fine-tuned on WikiText-103 (English) and mGPT on CC-100 Hindi; Llama-3.2-3B was used with its native weights for both languages. Applying three methodological controls - bidirectional arc coverage, syntactic-head isolation, and root-verb residualization - all four experiments confirm H1: a significant negative surprisal-confidence correlation (GPT-2 EN: $r=-0.4049$; GPT-2 HI: $r=-0.3684$; Llama EN: $r=-0.1420$; Llama HI: $r=-0.1386$; all $p<.001$). Disambiguation behaviour differs: GPT-2 shows a confidence crash with incomplete recovery (structural poisoning), while Llama-3.2-3B shows a spike in English and partial crash in Hindi, suggesting a distinct reanalysis mechanism in larger models.

1. Motivation and Research Problem

Large language models trained purely on next-token prediction encode rich syntactic structure in their attention patterns (Tenney et al., 2019; Clark et al., 2019). A central open question is whether this implicit syntax is *robust* under processing load or *fragile*, collapsing when the model encounters structurally ambiguous input - and whether robustness scales with model size.

Garden-path sentences force structural reanalysis at a disambiguating word, producing a sharp surprisal spike. We ask: does the attention weight LLMs' syntactic heads assign to the correct UD dependency partner decrease at high-surprisal positions? We extend this question across **two model families** (GPT-2 vs. Llama-3.2-3B) and **two languages** (English SVO vs. Hindi SOV), enabling the first systematic cross-model, cross-language comparison of syntactic attention fragility.

Primary Objective: Determine whether surprisal–confidence coupling is (i) consistent across model scale and (ii) consistent across typologically distinct languages, after correcting for methodological confounds.

2. Hypotheses and Predictions

H1 - Structural Breakdown. Token surprisal is defined as $\text{Surprisal}(w_t) = -\log P(w_t \mid w_2, \ldots, w_{t-1})$. We predict $r < 0$ in all four experiments: higher surprisal reduces syntactic-head attention confidence.

H2 - Disambiguation Crash and Recovery. Confidence will reach its minimum at the disambiguation token. Sub-hypotheses: (a) *Full Recovery*; (b) *Structural Poisoning* - confidence stays depressed.

H3 - Model-Scale Effect. Larger models (Llama-3.2-3B) show weaker $|r|$ than smaller models (GPT2), reflecting more distributed and robust syntactic representations.

3. Methods

Data. *English:* 1,064 garden-path sentences in CoNLL-U/UD format (GPT-2: 7,148 structural tokens; Llama: 6,602 structural tokens after alignment filtering). Mean sentence length = 23.4 tokens. *Hindi (HDTB):* 1,364 sentences, 7,676 structural tokens in both models. UD relations used: nsubj, obj, obl, ccomp, csubj, acl, xcomp, root, iobj.

Models. *GPT-2 (117M):* fine-tuned on WikiText-103 (English, 3 epochs, $lr=5\times10^{-5}$) and mGPT (1.3B) fine-tuned on CC-100 Hindi (500K sentences, 3 epochs, $lr=2\times10^{-5}$). mGPT's SentencePiece tokenizer handles Devanagari; GPT-2's BPE does not. *Llama-3.2-3B (3B):* used with native weights for both English and Hindi; 4-bit quantized (NF4) for memory efficiency. Both families return per-layer attention tensors.

Challenges to test our hypothesis and their fixes:

Fix 1 - Bidirectional Arc Coverage. GPT-2's causal mask blocks 63% of English UD arcs and 94.6% of Hindi UD arcs. For right-pointing arcs we reverse the query direction (head attends back to dependent). This is applied identically to both models.

Fix 2 - Syntactic Head Isolation. For each (layer, head) pair, compute fraction of tokens where argmax attention points to the correct UD head. Retain only pairs exceeding random baseline + 2□. GPT-2 has 144 heads (threshold □ 0.068); Llama has 672 heads (threshold □ 0.017). This yields 10 heads for GPT-2 and 10–18 heads for Llama.

Fix 3 - Root-Verb Residualization. We residualize both surprisal and confidence on binary `is_root` via OLS before computing Pearson r and Spearman □. For H2, confidence is compared across pre/at/post disambiguation windows using paired comparisons.

4. Results

4.1 Surprisal-Confidence Correlation (H1).

All four experiments confirm H1 (Figure 1, Table 1). GPT-2 shows stronger effects (EN: r=−0.4049; HI: r=−0.3684) than Llama (EN: r=−0.1420; HI: r=−0.1386), confirming H3: larger models exhibit weaker coupling between surprisal and syntactic attention.

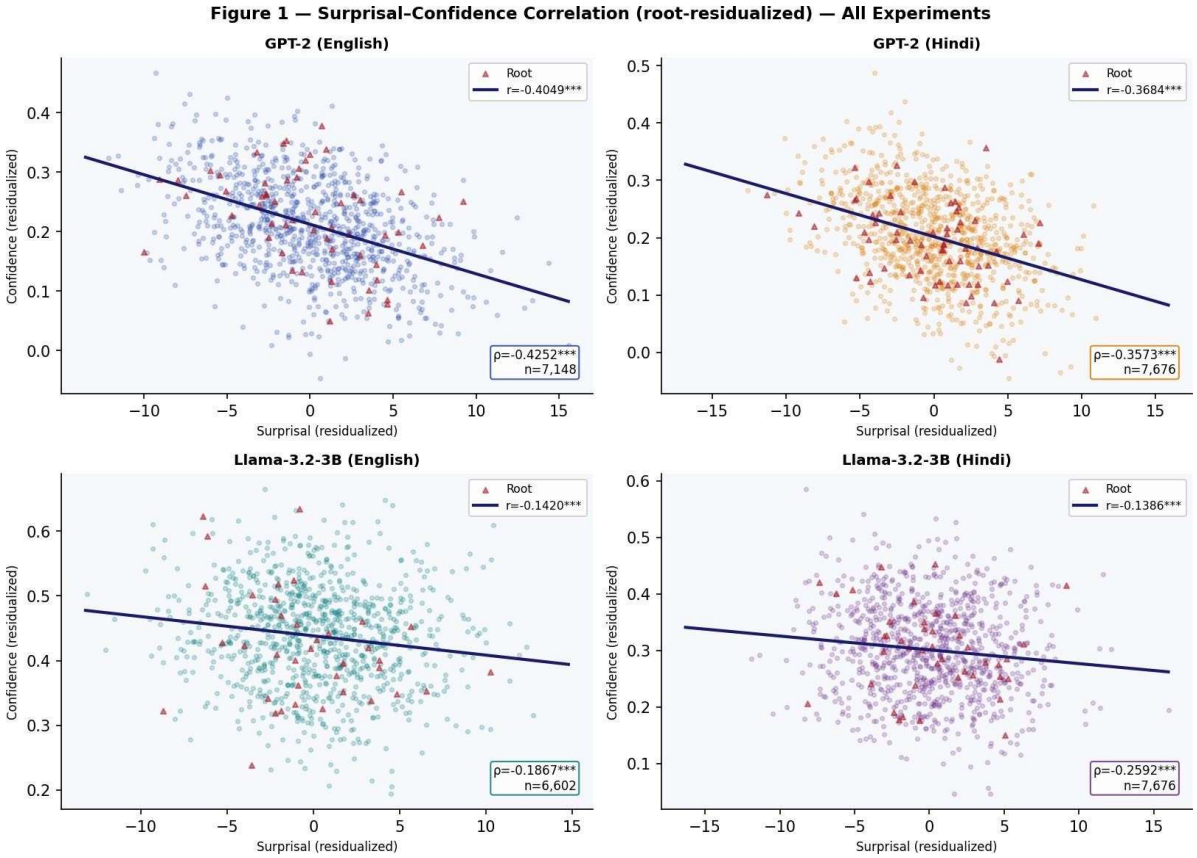

Figure 1. Root-residualized surprisal-confidence scatter for all four experiments. All negative slopes are significant (p<.001). GPT-2 shows stronger effects than Llama.

Experiment	Model	Lang	Pearson r	Spearman □	n	Syn. Heads
GPT-2 English	GPT-2	EN	−0.4049 ***	−0.4252 ***	7,148	10/144
GPT-2 Hindi	mGPT	HI	−0.3684 ***	−0.3573 ***	7,676	10/144

Llama-3.2-3B Eng.	Llama	EN	−0.1420 ***	−0.1867 ***	6,602	10/672
Llama-3.2-3B Hin.	Llama	HI	−0.1386 ***	−0.2592 ***	7,676	18/672

Table 1. Surprisal–confidence correlation across all four experiments. *** p<.001.

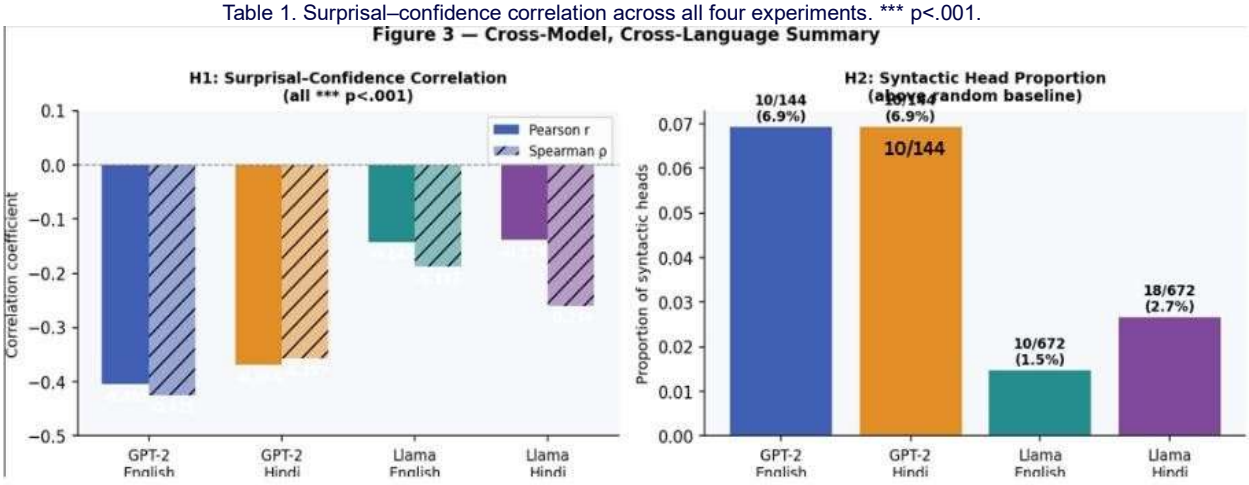

Figure 3. Left: Pearson r and Spearman across experiments (more negative = stronger effect). Right: Proportion of syntactic heads identified above random baseline.

4.2 Disambiguation Crash and Recovery (H2).

GPT-2 shows a confidence *crash* at disambiguation in both languages with incomplete recovery (structural poisoning). Llama-3.2-3B diverges: English shows a confidence *spike* at disambiguation (+0.079) before dropping post-disambiguation, while Hindi shows a mild crash (−0.035) with partial recovery (Figure 2). This inverted profile in Llama English suggests larger models engage additional reanalysis resources at hard structural boundaries.

Figure 2. Disambiguation crash and recovery across all experiments. GPT-2 crashes; Llama-EN spikes; Llama-HI shows partial crash.

Experiment	Pre	At	Post	Crash	Recovery	Pattern
GPT-2 English	0.221	0.138	0.204	−0.083	+0.066	Poisoning
GPT-2 Hindi	0.202	0.097	0.201	−0.105	+0.104	Poisoning
Llama-3.2-3B Eng.	0.456	0.535	0.423	+0.079	−0.112	Spike→Drop
Llama-3.2-3B Hin.	0.309	0.274	0.295	−0.035	+0.021	Partial Rec.

Table 2. Disambiguation crash and recovery across all experiments.

4.3 Syntactic Head Localization (Figure 5).

GPT-2 concentrates syntactic signal in Layer 4 (English) and Layer 3 (Hindi). Llama localizes syntactic heads in early layers (0-5) with sparse mid-to-late clusters (layers 12-14, 20-22), consistent with transformers encoding syntax in lower-mid layers (Tenney et al., 2019). The proportion of syntactic heads is smaller in Llama (10-18/672 □ 1.5-2.7%) than GPT-2 (10/144 □ 6.9%), confirming sparser encoding at larger scale.

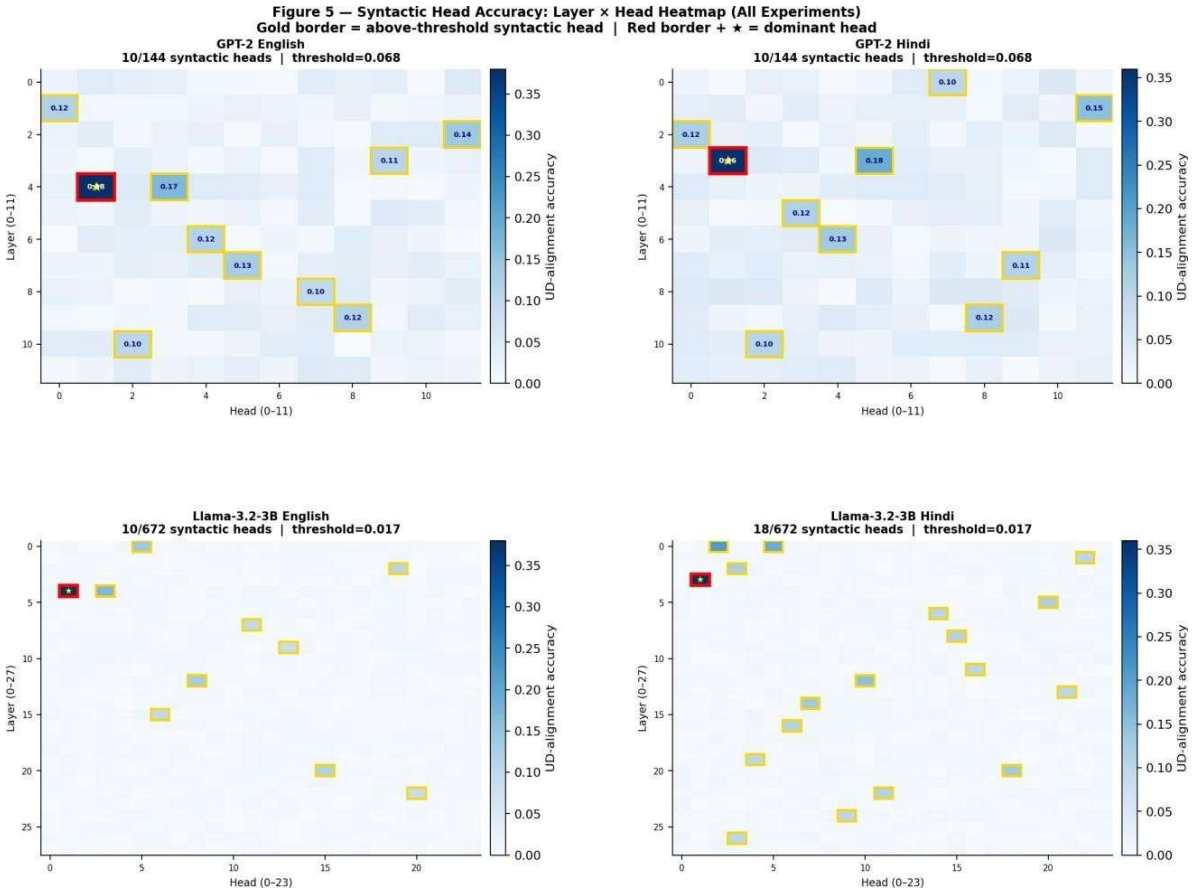

Figure 5. Layer × Head syntactic accuracy heatmap for all four experiments. Gold borders = above-threshold heads; Red border + ★ = dominant head.

4.4 Confidence by Dependency Relation (Figure 4).

Both models show consistent ranking across languages: **iobj** and **obl** suffer the most disruption, while **root** and **ccomp** show the highest confidence in GPT-2 Hindi (root=0.285, ccomp=0.277). In GPT-2 English, **iobj** leads with the highest confidence (0.455), while **ccomp** (0.080) and **obl** (0.093) remain the weakest - consistent with their structural complexity in SVO word order. Notably, Llama assigns dramatically higher confidence to **xcomp** (0.694 in Hindi) than any other relation across all four panels, while **ccomp** remains the weakest in Llama Hindi (0.120) - suggesting Llama preferentially tracks open clausal complements, consistent with its instruction-tuning objective. In Llama English, core argument relations **iobj** (0.666) and **nsubj** (0.621) show the strongest syntactic tracking, whereas **ccomp** (0.273) and **obl** (0.285) exhibit the

most breakdown, mirroring the Hindi pattern and reinforcing that clausal and oblique dependencies pose the greatest processing challenge across both languages and architectures.

Figure 4 — Mean Attention Confidence by Dependency Relation

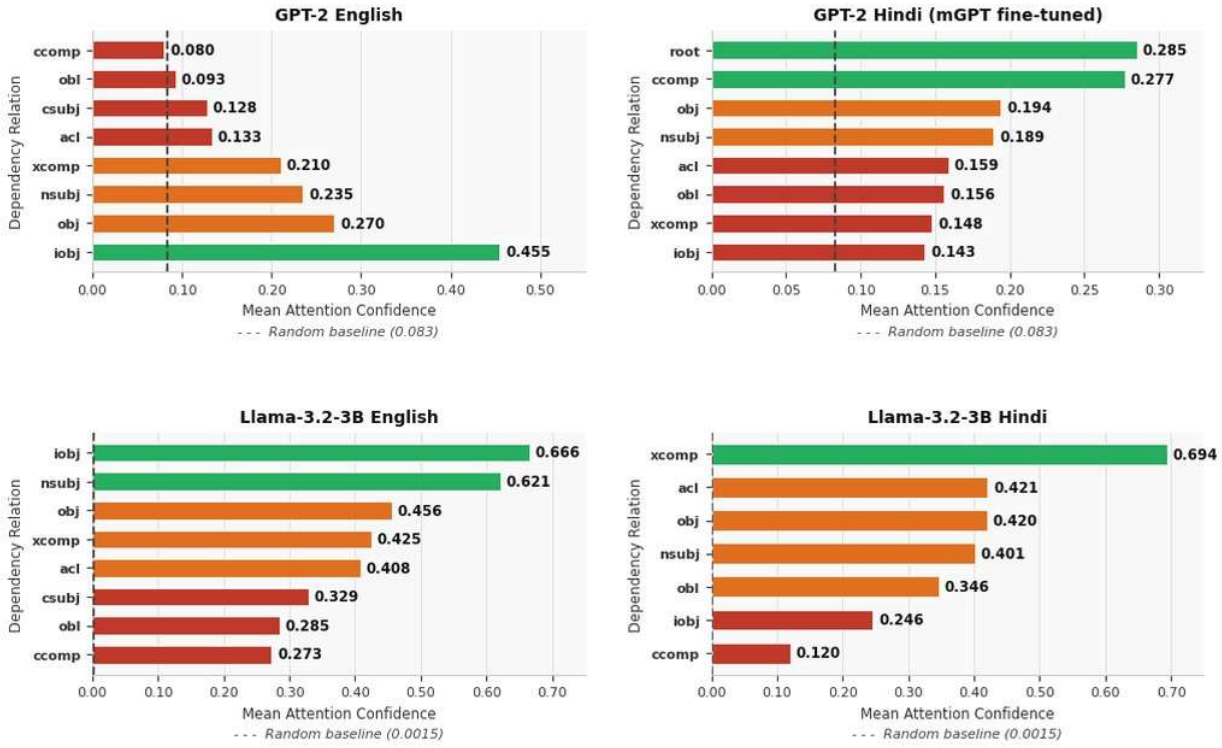

Figure 4. Mean attention confidence by dependency relation: GPT-2 English and Hindi vs Llama-3.2-3B English and Hindi. Dashed = random baseline

5. Theoretical Implications

5.1 Surprisal and Syntax are Load-Sensitive Across Models. All four experiments confirm that syntactic attention is load-sensitive, consistent with Surprisal Theory. The effect is real and not an artifact of architecture or language, since it replicates across GPT-2, Llama, English, and Hindi.

5.2 Model Scale Reduces but Does Not Eliminate Coupling (H3 Confirmed). GPT-2 shows r □ −0.40 while Llama shows r □ −0.14. Larger models encode syntax more robustly - distributing structural signal across more heads and layers - but syntactic attention remains coupled to lexical surprisal. This suggests distributional learning alone cannot fully decouple structural and predictive processing.

5.3 Good-Enough Parsing and its Model-Specific Breakdown. GPT-2's structural poisoning pattern (incomplete recovery) mirrors human 'good-enough parsing'. Llama's inverted English profile (spike at disambiguation) may reflect its broader pre-training engaging additional computation at structurally hard points - a form of effort allocation rather than collapse. The Hindi partial crash in Llama bridges these two patterns, suggesting language typology interacts with model capacity.

5.4 Cross-Linguistic Generalization. The negative surprisal-confidence correlation holds for both SVO English and SOV Hindi across both models. Hindi's richer morphology provides structural cues that partially compensate for causal-mask incompatibility, explaining the smaller magnitude difference between languages compared to the model-scale difference.

5.5 Limitations. (i) Bidirectional arc coverage is asymmetric vs. a true encoder. (ii) Attention weights are proxies for syntactic processing. (iii) GPT-2 and Llama have different tokenizers and pre-training corpora, partially confounding model-size comparisons. (iv) Hindi Llama analysis uses the multilingual model without Hindi-specific fine-tuning.

References

Christianson, K., et al. (2001). Thematic roles assigned along the garden path linger. *Cognitive Psychology*, 42, 368–407.

Clark, K., et al. (2019). What does BERT look at? An analysis of BERT's attention. *BlackboxNLP*, 276–286.

Ferreira, F., et al. (2002). Good-enough representations in language comprehension. *Current Directions in Psychological Science*, 11, 11–15.

Frazier, L., & Fodor, J. D. (1978). The sausage machine: A new two-stage parsing model. *Cognition*, 6(4), 291–325.

Futrell, R., Gibson, E., & Levy, R. P. (2020). Lossy-context surprisal. *Psychological Review*, 127(6), 1065–1100.

Hale, J. (2001). A probabilistic Earley parser as a psycholinguistic model. *NAACL*, 159–166.

Hewitt, J., & Manning, C. D. (2019). A structural probe for finding syntax in word representations. *NAACL-HLT*.

Jawahar, G., et al. (2019). What does BERT learn about the structure of language? *ACL*, 3651–3657.

Levy, R. (2008). Expectation-based syntactic comprehension. *Cognition*, 106(3), 1126–1177.

Nivre, J., et al. (2020). Universal Dependencies v2. *LREC*.

Radford, A., et al. (2019). Language models are unsupervised multitask learners. *OpenAI Blog*.

Tenney, I., et al. (2019). BERT rediscovers the classical NLP pipeline. *ACL*, 4593–4601.

Touvron, H., et al. (2023). Llama 2: Open foundation and fine-tuned chat models. *arXiv:2307.09288*.

Vaswani, A., et al. (2017). Attention is all you need. *NeurIPS*, 30.

Zeman, D., et al. (2022). Universal Dependencies 2.10. *LINDAT/CLARIAH-CZ*.

DATASET REFERENCES:

WikiText-103 was used for the fine-tuning of the gpt2 English
CC-100 Hindi 103 was used for the fine-tuning of the gpt2 hindi,
Garden-path and center-embedded sentences were extracted from raw Universal Dependencies CoNLL-U treebank files using a custom dependency-pattern heuristic script The pipeline requires no manual annotation and runs on both English and Hindi treebanks.

Universal Dependencies English Web Treebank (UD-EWT), Citation: Silveira et al. (2014). A gold standard dependency corpus for English. LREC 2014.
The Hindi corpus was extracted from the Hindi Dependency Treebank (HDTB) as distributed within Universal Dependencies Citation: Bhat et al. (2017). The Hindi/Urdu Treebank Project. Handbook of Linguistic Annotation, Springer.

The pipeline is at https://github.com/himank7737/A-Stress-Test-of-LLM-Syntactic-Heads

Appendix — Core Implementation

All experiments use Python 3.10+, Hugging Face Transformers, and the UD treebank.

A.1 GPT-2 Fine-tuning — English (WikiText-103)

```
from transformers import GPT2TokenizerFast, GPT2LMHeadModel, TrainingArguments, Trainer
from transformers import DataCollatorForLanguageModeling
from datasets import load_dataset
```

```
tokenizer = GPT2TokenizerFast.from_pretrained('gpt2')
tokenizer.pad_token = tokenizer.eos_token
model = GPT2LMHeadModel.from_pretrained('gpt2')
dataset = load_dataset('wikitext', 'wikitext-103-raw-v1')
def tokenize(ex): return tokenizer(ex['text'], truncation=True, max_length=512)
tokenized = dataset.map(tokenize, batched=True, remove_columns=['text'])
args = TrainingArguments(output_dir='./gpt2_en', num_train_epochs=3,
```

```
    per_device_train_batch_size=8, gradient_accumulation_steps=4,
    learning_rate=5e-5, fp16=True, eval_strategy='steps', eval_steps=500)
trainer = Trainer(model=model, args=args,
    train_dataset=tokenized['train'], eval_dataset=tokenized['validation'],
    data_collator=DataCollatorForLanguageModeling(tokenizer, mlm=False))
trainer.train(); trainer.save_model('./gpt2_en_finetuned')
```

A.2 mGPT Fine-tuning — Hindi (CC-100)

```
from transformers import AutoTokenizer, AutoModelForCausalLM
from datasets import load_dataset, Dataset

tokenizer = AutoTokenizer.from_pretrained('sberbank-ai/mGPT')
model = AutoModelForCausalLM.from_pretrained('sberbank-ai/mGPT')
raw = load_dataset('cc100', lang='hi', split='train', streaming=True)
texts = [ex['text'] for i, ex in enumerate(raw) if i < 500_000]
dataset = Dataset.from_dict({'text': texts}).train_test_split(test_size=0.01)
def tokenize_hi(ex): return tokenizer(ex['text'], truncation=True, max_length=512)
tokenized = dataset.map(tokenize_hi, batched=True, remove_columns=['text'])
args = TrainingArguments(output_dir='./mgpt_hi', num_train_epochs=3,
    per_device_train_batch_size=4, gradient_accumulation_steps=8,
    learning_rate=2e-5, fp16=True, gradient_checkpointing=True)
trainer = Trainer(model=model, args=args,
    train_dataset=tokenized['train'], eval_dataset=tokenized['test'],
    data_collator=DataCollatorForLanguageModeling(tokenizer, mlm=False))
trainer.train(); trainer.save_model('./mgpt_hi_finetuned')
```

A.3 Llama-3.2-3B Loading (4-bit quantized)

```
from transformers import AutoTokenizer, AutoModelForCausalLM, BitsAndBytesConfig
import torch

bnb = BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_use_double_quant=True,
    bnb_4bit_quant_type='nf4', bnb_4bit_compute_dtype=torch.float16)
tokenizer = AutoTokenizer.from_pretrained('meta-llama/Llama-3.2-3B', token=HF_TOKEN)
model = AutoModelForCausalLM.from_pretrained('meta-llama/Llama-3.2-3B',
    output_attentions=True, quantization_config=bnb, device_map='auto', token=HF_TOKEN)
model.eval()
# n_layers=28, n_heads=24, total=672 heads
```

A.4 Surprisal Extraction (shared across models)

```
def compute_surprisal(text, tokenizer, model, device):
    enc = tokenizer(text, return_tensors='pt').to(device)
    with torch.no_grad():
        out = model(**enc, output_attentions=True)
    log_probs = torch.log_softmax(out.logits[0].float(), dim=-1)
    surp = {t: -log_probs[t-1, enc['input_ids'][0,t]].item() / math.log(2)
            for t in range(1, enc['input_ids'].shape[1])}
    return surp, out.attentions
```

A.5 Fix 1 — Bidirectional Arc Coverage

```
def arc_attention_weight(attn, dep_pos, head_pos):
    # left arc: dep attends to head (standard causal)
    # right arc (Fix 1): head attends back to dep
    src = dep_pos if head_pos < dep_pos else head_pos
    tgt = head_pos if head_pos < dep_pos else dep_pos
    return float(attn[:, :, src, tgt].mean())
```

A.6 Fix 2 — Syntactic Head Identification

```
def identify_syntactic_heads(all_conf_matrices, n_layers, n_heads, sigma=2.0):
    acc = np.zeros((n_layers, n_heads))
    for mat in all_conf_matrices:
    best = np.unravel_index(np.argmax(mat), mat.shape)
    acc[best] += 1
acc /= len(all_conf_matrices)
baseline = 1.0 / (n_layers * n_heads)
threshold = baseline + sigma * np.std(acc)
return [(l,h) for l in range(n_layers)
        for h in range(n_heads) if acc[l,h] > threshold], acc, threshold
```

A.7 Fix 3 — Root-Verb Residualization + Correlation

```
def residualize(y, covariate):
    X = np.column_stack([np.ones(len(y)), np.array(covariate, float)])
    b = np.linalg.lstsq(X, np.array(y, float), rcond=None)[0]
    return np.array(y, float) - X @ b
```

```
surp_res = residualize(df['surprisal'], df['is_root'].astype(float))
conf_res = residualize(df['conf_max'], df['is_root'].astype(float))
r, p     = pearsonr(surp_res, conf_res)
rho, _ = spearmanr(surp_res, conf_res)
```

A.8 Recovery Analysis

```
WINDOW = 4
def recovery_analysis(records):
    conf_by_offset = {o: [] for o in range(-WINDOW, WINDOW+1)}
    for r in records:
        o = r.get('offset')
        if o is not None and not np.isnan(r['conf_max']):
            conf_by_offset[int(o)].append(r['conf_max'])
    return {o: np.mean(v) for o, v in conf_by_offset.items() if v}
# Pre = offset -1, At = offset 0, Post = offset +1
```

A.9 Model Configuration Summary

Parameter	GPT-2 English	GPT-2 Hindi	Llama EN	Llama HI

Base model	GPT-2 (117M)	mGPT (1.3B)	Llama-3.2-3B	Llama-3.2-3B
Fine-tune corpus	WikiText-103	C-100 Hindi (500K sent.)	-	-
Fine-tune epochs	3	3	—	—
Architecture	12L × 12H	12L × 12H	28L × 24H	28L × 24H
Total heads	144	144	672	672
Tokenizer	BPE (GPT2Fast)	SentencePiece (mGPT)	BPE (Llama)	BPE (Llama)
Syn. heads	10 / 144 (6.9%)	10 / 144 (6.9%)	10 / 672 (1.5%)	18 / 672 (2.7%)
Dominant head	Layer 4, Head 1	Layer 3, Head 1	Layer 4, Head 1	Layer 3, Head 1
Pearson r	−0.4049 ***	−0.3684 ***	−0.1420 ***	−0.1386 ***
Spearman □	−0.4252 ***	−0.3573 ***	−0.1867 ***	−0.2592 ***
Structural tokens	7,148	7,676	6,602	7,676
Disambiguation	Crash → Partial rec.	Crash → Partial rec.	Spike → Drop	Crash → Partial rec.